

Lab experience 3: LabVIEW as a Function Generator

For Essick, 4th edition

Objective

- Learn programming skills to create a LabVIEW-based function generator
- Evaluate the limits on signal output through a USB-600x device.

Reading & new concepts

Hardware

Each USB-6001, 6008 and 6009 device has two 12-bit digital-to-analog converters (DACs) that change their output value(s) when LabVIEW sends the appropriate command. The DAQ Assistant and VI will be used to generate signals, using a selected amount of channels and specified sample mode. If you want the DAQ Assistant to write to two channels (or whatever amount of channels you've selected), you need to send an array of two values into DAQ. If it is set for two channels and you want to hold one value constant, then you need to send that channel the same value as you sent last time.

The USB-6008 and USB-6009 have only one output voltage range, 0 to 5 V. Unlike the analog input, there is no 'differential' output; both channels have the same GND reference that is defined as 0 V. The output voltages are updated on demand, such that there is no 'N samples' option. The device specifications state that the maximum output rate is 150 samples/second, but you can send commands to the device faster than that to observe its behavior. A similar DAQ model, the USB-6001, allows us to send an entire waveform at once as a sampling rate and array of signal values; the hardware then outputs the signal based on its internal clock at a rate up to 5000 samples per second.

Programming

Read sections 4.1 - 4.2 if you need an introduction to Formula nodes. The book refers to a now (unfortunately) obsolete programming feature called MathScript. Instead, we will use Formula nodes, which evaluates mathematical formulas and expressions similar to C on the block diagram.

Read section 6.4 (or 5.10 of 3rd edition), DC Voltage Source. You should understand the steps, but the programming exercise itself is optional. Note the use of the STOP input to the DAQ Asst in the figure on page 230 (p.205 in 3rd edition). This feature is important because, as you found in Lab 2, opening and closing the DAQ task takes time and can slow down execution. Note also that the DAQ changes its output *only when you tell it to* (or unplug it) so if it is going along at 3.14 V it will *stay* at 3.14 V when you stop the VI if the VI does not send a "0 V" signal first. Therefore, it is highly recommended that you put a DAQ Asst outside the loop to set the output to zero before the VI terminates.

Pre-lab programming and calculations

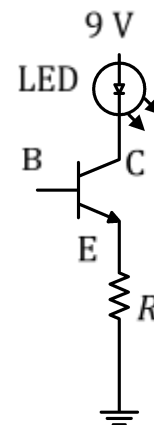
If possible, complete the calculations below before your lab section, to enable you to focus on circuit building and system testing. **Creating a draft VI is optional.** Show the instructor your VIs and calculations, then proceed to circuit building. Of course, if you can't get to it before your section, then you'll start with those tasks.

1. Controlling the light level from an LED using LabVIEW and a VCCS (pre-lab)

This activity uses a voltage-controlled current source (VCCS) to regulate the light produced by a light-emitting diode as a function of DAQ output voltage. It is necessary because the USB-600x does not have the current capacity to drive an LED.

Do not connect an LED directly to the analog output terminals!

The simplest VCCS uses a resistor and 2N3904 or similar BJT (bipolar junction transistor). In the figure at right, the DAQ output connects to the base B, and $V_E \approx V_B - 0.6$ V. Let's start with some simple engineering calculations to select our components.



- a) Assume that we would like to operate the LED with currents $0 < I < 50$ mA. Choose a resistance for your VCCS that will ensure the LED current limit is not exceeded when the DAC voltage is at its maximum. Show your calculations.

- b) Determine the power that your resistor will dissipate when the DAC voltage is at its maximum.

- c) Find the data sheet for your transistor, for example [G:\Datasheets\2N3904.pdf](#) or [2N3904 from Digikey](#). Look for I_C in the Absolute Maximum Ratings table, and confirm that the transistor's current capacity is sufficient, *i.e.* greater than 50 mA.

- d) Find the transistor's DC Current Gain, h_{FE} , in the Electrical Characteristics table. This gain is the ratio of current through the emitter to current through the base. Verify that when the emitter current is 50 mA, the current from the DAC to the base will not exceed 5 mA, which is the maximum DAC drive current.

2. Connect or simulate a DAQ module

- If you are in lab, connect a USB-6008 or similar device to your computer.
- If you do not have a physical DAQ module, create a simulated device.
 - Open NI-MAX (NI measurement and automation explorer).
 - In the My System tree, right click on Devices and Interfaces, click Create New >> Simulated NI-DAQmx Device or Modular Instrument. Expand USB DAQ, select USB-6008, and click OK. You can now create a VI using DAQmx functions as if you have USB-6008 hardware attached to your computer.

3. Use LabVIEW to read a saved bio-signal from disk and regenerate the signal

a) Open the file 468_SignalRead_OutputOnce.vi, and use it to read, display, and output a signal that is saved as a LabVIEW measurement (.lvm) file. Observe the **signal on the oscilloscope**. The sampling rate has been included in the signal file name so you can set the loop period appropriately.

b) Save the 468_SignalRead_OutputOnce.vi VI as **"468 Lab3_nestedloop_yourname.vi"** and modify it so that the signal repeats continuously. This can be done by creating a while loop around the for loop, but leaving the File Read function outside the while loop.

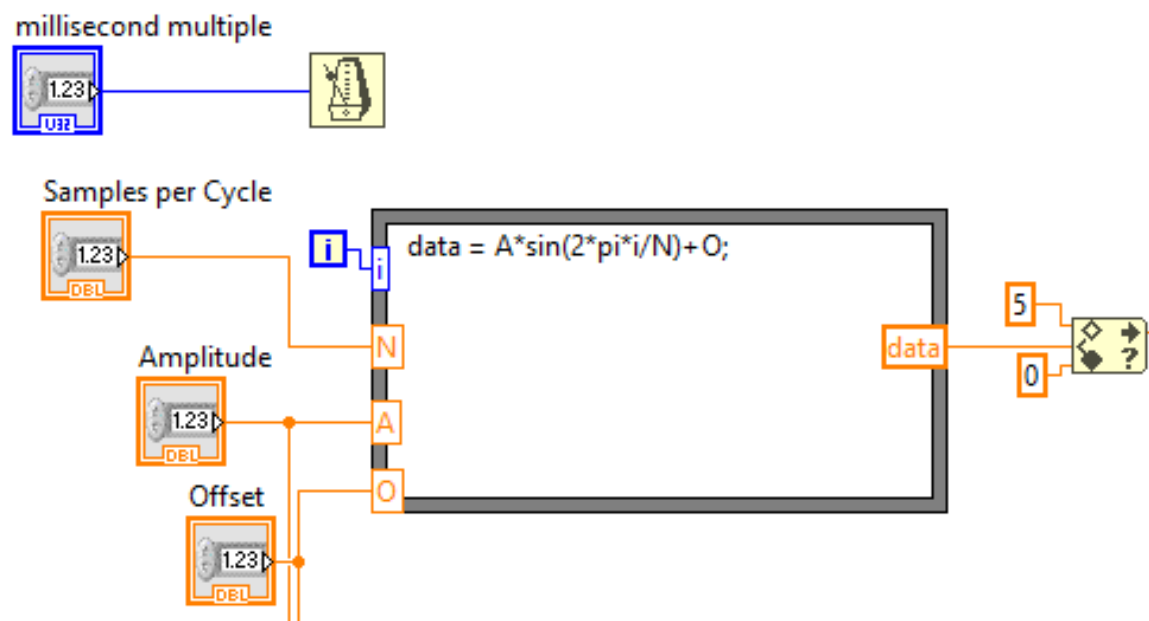
c) Add a DAQ assistant (Measurement I/O > DAQmx > DAQ assistant > Generate Signals > Analog Output > Voltage > Select one channel; Generation Mode > 1 Sample (on demand)) outside of the while loop and wire a "0" constant into the data terminal. This will set the output voltage of the DAQ to zero when the loop is stopped. Add a stop control to the second DAQ assistant and set it to true. Remember from Lab 2; this is to enhance the performance of our VI. **Make sure the DAQ Assistant within the loop is taking in one sample (on demand) and that the physical channel of the output is correct. YOU SHOULD ONLY HAVE ONE OUTPUT IN YOUR DAQ ASSISTANT BECAUSE THERE IS ONLY ONE SIGNAL.**

d) Run the VI to observe that the signal repeats until you click Stop. PLOT YOUR SIGNAL ON THE OSCILLOSCOPE TO MAKE SURE YOUR DAQ SETUP WORKS

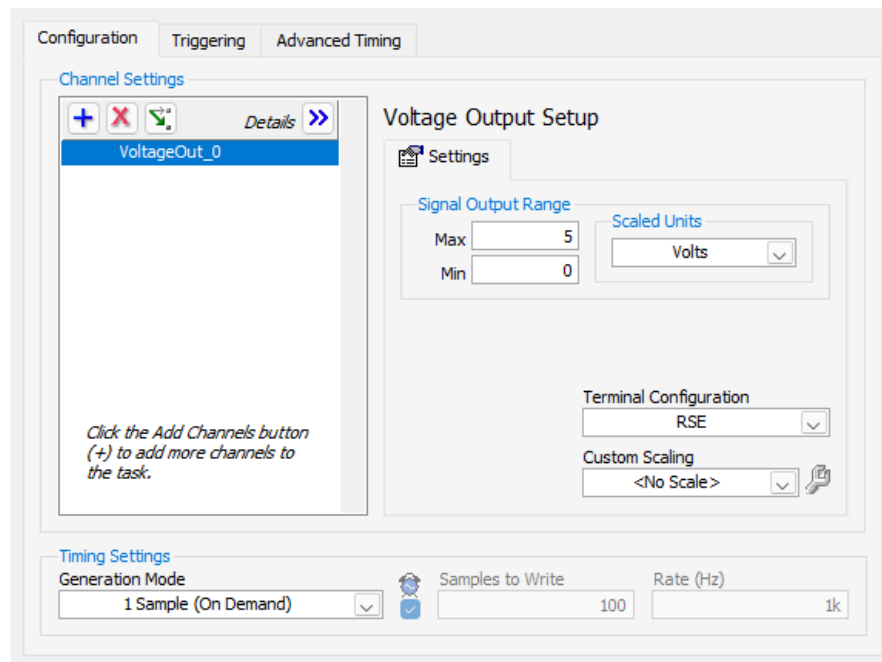
e) *Optional*. Notice that the for-loop continues to execute even when you toggle the Stop button. To allow the user to break out from the for-loop, we need to change the for-loop into a conditional for-loop. Right click a vertical side of the for-loop and select "Conditional Terminal". You will notice a stop icon appears. Move the icon button into the for-loop. Wire the icon to the stop (T) of the DAQ assistant and to the stop icon of the while-loop. **You will get an error at the tunnel from the for-loop to the while-loop.** Right click the tunnel and change the mode to last value. (tunnel>tunnel mode>Last Value). Run the VI again and make sure that you can now terminate it immediately. **If you choose not to do 3.d. you still need to wire the while loop's conditional terminal to a control to run your VI.**

4. Create a LabVIEW-based **sine** generator

- Create a new VI. Create a while-loop. Add a 'Wait until next ms multiple' and set the default for the front-panel control to 50 ms.
- Add a formula node. Create front panel controls for the following input variables: amplitude, offset, samples per cycle, and iterations of the sine wave. Pass these variables into the formula node by right clicking the formula node and "Add input".
- Similarly, add an output for data that will be passed from the formula node to an 'In range and coerce' function. This function ensures the voltage value is between 0 and 5 V. The coerce function is in Programming >> Comparison.
- Write an equation for a sine function using the variables you've created in the formula node. **Make sure you add a semicolon at the end of the equation.** Your VI may look something like this:



e) Add a DAQ assistant (Measurement I/O > DAQmx > DAQ assistant > Generate Signals > Analog Output > Voltage > select one channel > generation mode > 1 sample on demand) and wire the data from the coerce function into the DAQ assistant.



f) Add an error out indicator and wire the status to the while loop's stop button. Remember to also wire the stop (T) to the stop button. Recall what you did for Lab 2.

g) Plot the data from the coerce function onto a waveform chart. Set the amplitude and offset to 0 and run the VI to observe the waveform on the chart. Repeat with the offset at 2.4 V, then at 4.8 V. Adjust the front panel controls until you are satisfied with the plotted waveform. Congratulations! You've created a function generator.

h) *Optional.* You may find it helpful to use property nodes to force the chart to fit the signal you've generated. Example: Y-scale minimum and maximum based on the signal's amplitude.

Save this VI as Lab3_sinegenerator_yourname.vi so that you can use it later.

5. Create a LabVIEW-based **square wave** generator

Let's define a square wave as a function that alternates between some positive constant and zero, with the same amount of time in each phase. Because we are using a USB-powered DAQ module, the positive constant can be up to 5 V.

There are many ways to generate a square wave, including but not limited to the following options (that **aren't** instructions):

- Use the `square(x)` function in Mathematics >> Elementary >> Gating, and scale the loop index as desired to change the square wave period;
- Create the Waveform Simulator introduced in section 4.3 of the text.

However, the most efficient way is to provide only one output value per phase, telling the DAC what the output should be only when you want it to change. In this case, you could apply the quotient function, which looks like $\%2$ in the Formula node, to the loop iteration counter i , to produce an alternating 0-1-0-1-0... which you can multiply by a constant to get whatever amplitude you want. You can accomplish the same thing other ways with or without the Formula node if you prefer.

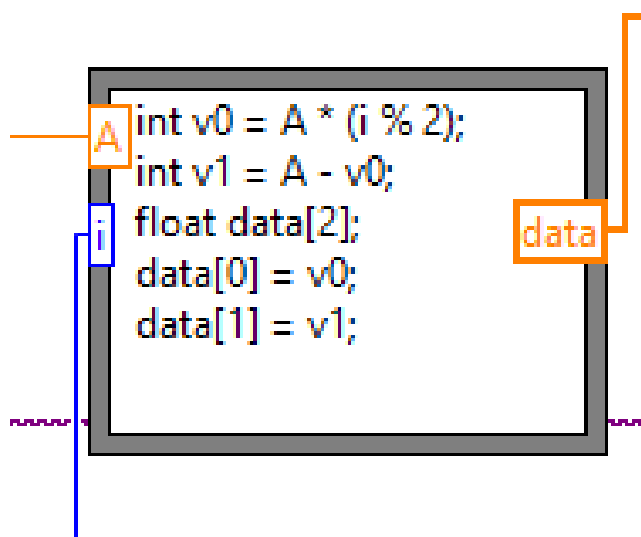
a) Create a new VI. Create a while-loop. Add a 'Wait until next ms multiple' and set the default for the front-panel control to 50 ms.

b) Add a formula node. Create front panel controls for the following input variables: amplitude and iterations of the square wave. The goal is to just alternate between the positive and zero values. For example, if the amplitude is a , then the output signal would be ...0, a , 0, a ... with a transition every time the while loop does one iteration. This means two iterations per period. Pass these variables into the formula node by right clicking the formula node and "Add input".

c) Similarly, add an output for data that will be passed from the formula node to an 'In range and coerce' function. This function ensures the voltage value is between 0 and 5 V. The coerce function is in Programming >> Comparison.

d) Write an equation for your square wave generator so that it creates two channels that are opposite in phase. That is, if the amplitude is a , then AO0 (analog output channel 0) would be 0, a , 0, a ... and AO1 would be a , 0, a , 0... with both waves having a transition (but in the opposite direction) every time the while loop does one iteration. Note that it is easy to produce the second signal as $v_1(t) = a - v_0(t)$.

e) In your formula node, write the code to combine $v_0(t)$ and $v_1(t)$ into a 1x2 array, which LabVIEW will interpret as a 1D array. One method to make this happen:



Add an analog output DAQ assistant (see part 4.) to wire the data into and make sure to select/add two channels, not one.

- Interestingly, the Waveform Chart will plot both channels from the 1-D array as a single curve because it concatenates them into a long 1-D array. To get two curves we need a 2-D array, which can be created from the 1-D array with the Build Array function. Right click on the wire, click Insert >> Array Palette >> Build Array.

d) Verify your VI's operation by observing both channels on the chart display, and save your VI for use with the LED driver circuit.

Save this VI as Lab3_squarewave_yourname.vi so that you can use it later.

In-lab circuits and testing

If you haven't already, connect a USB-6008 or similar device to your computer. For each VI that you created before, you will need to set the physical channel you will be using as an output on the front panel.

6. Test your arbitrary signal generator

- a) Open the VI that you modified to display the EKG signal continuously, and use an oscilloscope to observe the signal as you run the VI.
- b) Once you have stopped the VI, verify that the output from the DAQ module is 0 V.

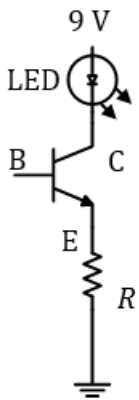
7. Test your LabVIEW-based sine generator

- a) Open your sine wave generator VI and update the USB-600x output channels.
- b) Set the amplitude and offset to 0 and measure the DAQ output voltage with a digital multimeter. Repeat with the offset at 2.4 V and 4.8 V. **Make sure you have samples per cycle set to a non-zero value for the VI to run.** Compare the observed error to the 'Accuracy' value for analog output shown in the Specifications section of the [USB-6008 web page](https://www.ni.com/docs/en-US/bundle/usb-6008-specs/page/specs.html) (<https://www.ni.com/docs/en-US/bundle/usb-6008-specs/page/specs.html>). Note that the values should be in volts or percent to compare the two.

Observed:

Published:

- c) Connect your DAQ output terminal to an oscilloscope and verify that the output is the desired sine wave for various values of samples per cycle and loop period, amplitude and offset.
- d) Decrease the samples per cycle and loop period until the transition from one output voltage to the next is clearly visible as a stairstep on the oscilloscope screen.
 - Estimate the rate at which the voltage changes from one level to the next (V/ms).
 - What, if anything, do you observe as you push the loop rate higher than 150 Hz?



3. Controlling the light level from an LED using LabVIEW and a VCCS (in lab)

a) Create a VCCS using the components that you selected, or the closest values that you have on hand. **Do not connect the LED directly to the DAQ's analog output without a series resistor!**

b) Connect the DAQ output to the base of the transistor and run your sine-generator VI to verify that the LED brightness varies sinusoidally.

Use the 5V output from the DAQ (if you can). If not, use 6V from the benchtop power supply. The 9V is to make sure you have enough voltage to run your circuit/use a 9V battery.

4. Alternating LED output using LabVIEW and two VCCS (in lab)

NOW THAT'S AN LED OF A DIFFERENT COLOR!

The circuit we are creating, with two alternating LEDs, is one essential component of the optical sensor in a pulse oximeter.

a) Build a second LED-BJT-Resistor circuit in parallel with the first. Test it with the output from your sine wave generator VI.

b) Open the 2-channel square wave generator VI and use its two outputs to control the two LED driver circuits. The two LEDs should alternate which one is on and which is off.

d) Using the oscilloscope, observe the transition timing as the AO0 and AO1 channels switch from high to low and vice versa.

- In our VI, it appears that the write operation to the DAQ should produce the transitions at AO0 and AO1 simultaneously. Do you see any delay from one channel to the other?

e) **Demonstrate your working circuit and software** to the world... or at least to the instructors.

f) **Save your creations for the next lab period, when you will measure the light you generated!**

Deliverables (graded on completeness):

- Lab3_squarewave_yourname.vi
- Demonstration

Debugging check-list. When your circuits do not work, ask yourself:

For LabVIEW:

- Is the sample per cycle too few? Is amplitude not a positive number? Is the offset too big?
- Is the stop button stuck at True? Make its default value False and set its mechanical action to “Latch when released” (or other latching mechanical action depending on your application).
- Are all DAQ Asst’s set up correctly? Do they have all the necessary channels?
- Does the voltage range of the waveform exceed the voltage limit (0 - 5 V) of our USB-powered DAQ module?
- Should it be a waveform graph or chart?
- Are things really connected on the block diagram?
- Have I tried the following debugging methods?
 1. While the VI is running, probe the wires on the block diagrams to see the data types and values.
 2. Use the Highlight Execution button (the lightbulb icon next to the run button) to visualize the data flow.
 3. On the block diagram, click Help >> Show Context Help. Hover the cursor over the function icon and wires to inspect if they are indeed the correct functions/data types.
 4. Sometimes a function icon can hide behind another function icon. Try moving the icons around to reveal anything out of sight.

For Circuits:

- Is the power supply output turned on?
- Is everything connected properly?
- Are all leads inserted nicely into the breadboard?
- Is the polarity of the LED correct? The large reflective cup (flat top) in the epoxy case is the cathode (-) lead.
- Is the BJT transistor connected properly? Take a look at the data sheet to confirm the terminals.
- Do the DAQ and the power supply share the same ground as they should?
- Use a voltmeter to check the following:
 - The LED voltage is high enough:
 - 2.0 – 2.2 for red
 - 2.2 – 2.4 for yellow
 - 2.4 – 2.6 for green
 - > 3.5 for blue.
 - The emitter-base voltage is around 0.6 V.

- The DAQ output is as expected

Catching up

The BIOE undergraduates have conducted a lab exercise in BIOEN 327 that passed a signal through the 'line out' port in the computer sound card, to the 'line in' port in the same or another computer. The sound card provides an inexpensive, high frequency signal input and output function, but it is worse than the USB-6008 at low frequencies (the sound card contains a high-pass filter) and has a smaller voltage range (the sound card is limited to ± 1 V). Anyone interested in exploring the sound card as an I/O port is welcome to do part 2 in the BIOEN 327 Intro to LabVIEW handout.

Optional reading

Explore the use of an XY graph as shown in section 4.3 and 4.4, noting that the input to a waveform graph is a cluster that bundles t_0 , Δt and an array of values $y(t)$, while the input to an XY graph is a cluster that contains an array of x values and a corresponding array of y values. Bundling is formally introduced in section 3.9.

Section 4.10 describes the use of control and indicator clusters. These clusters are optional (you could just connect the multiple individual wires from one icon to the next), but bundling reduces the total number of wires that need to go from one end of the block diagram to the other. On the other hand, bundling increases the number of icons, so there is some trade-off between the two options.

Sections 4.11 - 4.13 show how to edit terminals and icons. This process is necessary when you start grouping block diagram functions into sub-VIs.